FUJINET ENGINEERING SERIES

Platform Bring-Up Guide

Adding new platform support to FujiNet — from bus adapter to PIO to device firmware.

Worked example: the 8-bit PC ISA bus

The ESP32 + RP2350 tandem design
Hardware · PIO · Firmware · Client library

Revision 1 · June 2026
The FujiNet Project

About this guide

This manual guides a small team of hardware and firmware engineers through the complete process of bringing a new computer platform onto FujiNet using the **bus-interface tandem** design: a Raspberry Pi RP2350 that speaks the target machine's native bus, paired with an ESP32 that runs the FujiNet device firmware. The 8-bit IBM PC/XT ISA bus is used throughout as a concrete, end-to-end example because a prototype ISA adapter, GPIO map, and edge footprint already exist in the `fujiversal-pcb-prototype` repository.

Every register value, pin assignment, packet field, and source excerpt in this guide was transcribed from the live project sources listed on the copyright page of each repository, not from secondary documentation. Where the prototype is deliberately incomplete or risky, the text says so plainly.

Canonical sources

```
fujiversal                RP2350 bus-interface firmware — PIO, USB-CDC bridge, ROM emulation
fujiversal-pcb-prototype  Universal proto board, ISA / CoCo / MSX adapters, footprints
fujinet-lib-experimental  Host client — FujiBus framing over the I/O byte pipe
fujinet-firmware          ESP32 device firmware — lib/bus, lib/device, lib/media
fujinet-config            Host-side ROM / CONFIG image served by the RP2350
FEP-004 (wiki)            FujiNet serial-encapsulation protocol proposal
```

Contents

PART I

Orientation

What it means to bring up a platform, how the tandem design divides the work between two chips, and the FujiBus protocol that ties them together. Read this part before touching hardware.

CHAPTER 1

Introduction

FujiNet is a network and storage peripheral for retro computers. To a 1980s machine it looks like a fast disk drive, a printer, an RS-232 modem, a real-time clock, and a handful of other devices; behind that façade it is an ESP32 microcontroller with WiFi, an SD card, and a small army of internet protocol adapters. "Bringing up a platform" means making FujiNet appear as those familiar peripherals on a machine it has never run on before.

1.1 Three ways to bring up a platform

Historically FujiNet has been ported three different ways. Choosing the right one for your target machine is the first design decision, and it determines almost everything that follows.

Adapt an existing serial bus	The machine already has a multi-drop serial peripheral bus with a documented protocol. FujiNet simply becomes another device on that bus, speaking the native protocol directly on the ESP32.	Atari SIO, CoCo DriveWire
FEP-004 serial encapsulation	The machine has a UART or a simple serial link but no peripheral protocol. FujiNet defines its own framing — FEP-004 — over that link, and a small host-side driver speaks it.	RS-232, MSX (serial)
Bus-interface tandem	The machine has no serial bus at all — only a parallel CPU expansion bus (cartridge slot, ISA, S-100, Apple slot…). A second microcontroller, an RP2350, sits directly on that parallel bus and bridges it to the ESP32. This is the subject of this guide.	ISA (this guide), MSX (cartridge), CoCo (cartridge)

Table 1. The three FujiNet bring-up strategies. This guide covers the third.

The tandem design exists because parallel CPU buses are **fast and unforgiving**. A Z80 or an 8088 expects valid data on the bus within tens of nanoseconds of asserting a read strobe. An ESP32 running FreeRTOS and a WiFi stack cannot meet that deadline. The RP2350 can: its Programmable I/O (PIO) blocks are deterministic state machines that react to bus signals in single clock cycles, and its second core can be dedicated to the bus with interrupts disabled.

1.2 The division of labour

The single most important idea in this guide is how responsibility is split across the two chips. Internalise this and the rest of the manual is detail.

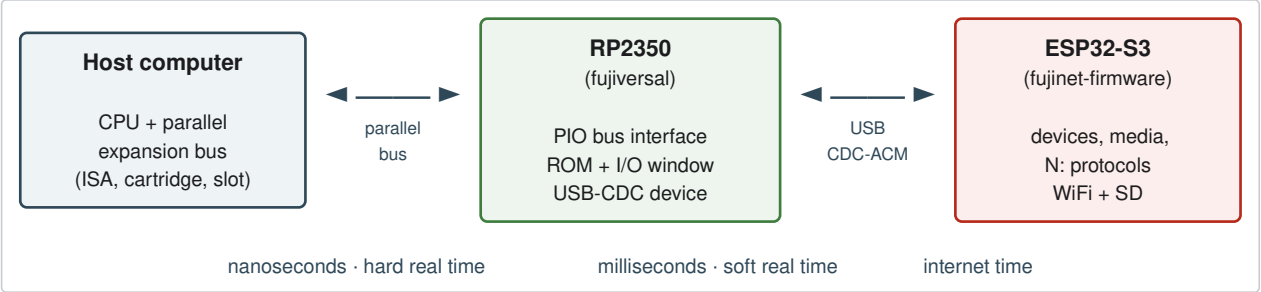

Figure 1. The tandem architecture. The RP2350 owns the time-critical parallel bus; the ESP32 owns devices and the network. They meet over a USB serial link carrying FujiBus packets.

- **The host** runs unmodified software. To it, FujiNet is a ROM in its address space plus a few I/O registers — nothing more exotic than a period-correct expansion card.
- **The RP2350** (the `fujiversal` firmware) emulates a ROM chip and a small bank of I/O registers on the host's bus. The ROM holds the host-side loader and CONFIG program. The I/O registers are a **byte pipe**: the host pushes and pulls bytes through them, and the RP2350 shuttles those bytes over USB to the ESP32. The RP2350 understands almost nothing about FujiNet devices — it is a transparent pipe with one exception (ROM bank-switching, covered in Chapter 8).
- **The ESP32** (the `fujinet-firmware`) runs the entire FujiNet device stack: the Fuji control device, virtual disk drives, the `N:` network device, printers, clock, CP/M, and the internet protocol adapters behind them. It receives FujiBus packets over the USB serial link exactly as if they had arrived on any other FujiNet transport.

IMPORTANT
Because the ESP32 sees a **serial stream of FujiBus packets**, a bus-based platform reuses the firmware's existing serial transport — the `rs232` bus class — almost unchanged. The genuinely platform-specific engineering concentrates in two places: the **RP2350 PIO program** (Part III) and the **host ROM + client library** (Part IV). The ESP32 side is mostly a build-system and pin-map exercise. This is the payoff of the tandem design, and a recurring theme of this guide.

1.3 What you will build

By the end of this guide you will have produced, for your target platform (ISA in the worked example):

1. A **bus adapter** — a small PCB that mates the universal prototype board to the machine's physical connector, with whatever level-shifting the bus voltage demands (Chapters 5–7).
2. A configured **prototype board** — jumpers set, test points identified, the two dev boards seated and powered (Chapter 5).
3. An **RP2350 PIO program** — a `boards/<platform>.pio` file that decodes your bus and implements the ROM + I/O-window byte pipe (Chapter 9).
4. **ESP32 firmware support** — a build target, a pin map, and (only if needed) new device or media classes (Chapters 11–14).
5. A **host ROM and client library** — the loader/CONFIG image the RP2350 serves, and the `fujinet-lib` bus backend that drives the byte pipe (Chapters 15–16).

1.4 Conventions

Inline code, signal names, register names, and file paths are set in `monospace`. Active-low signals are written `/IOR` or with an overline in schematics (KiCad renders these as `~{IOR}`). Hexadecimal is written `0x1234`; an I/O **port** address is also written `0x300`. GPIO numbers on the RP2350 are written `GP17`. Callout boxes carry four levels of emphasis:

NOTE
Background, rationale, or a pointer to a deeper source file.

TIP
A shortcut, a known-good value, or a debugging trick.

IMPORTANT
Something you must get right or the design will not work.

⚠ MAGIC SMOKE — HARDWARE HAZARD
A way to destroy hardware. Read these twice.

CHAPTER 2

System architecture

This chapter is the map. It names every board, chip, firmware image, and wire, and shows where each one lives in the repositories. Later chapters zoom into the boxes drawn here.

2.1 The physical stack

A bring-up rig is three boards (Figure 2 shows how they relate):

Waveshare Core2350B the RP2350 board (component `U1` on the prototype schematic, footprint `FujiNet:WaveShare-Core-RP2350B`). The RP2350B variant brings out up to 48 GPIO, which is what makes a 20-bit address bus plus 8-bit data plus control signals fit on one chip. It connects to the host bus through the prototype board, and to the ESP32 over USB.

Freenove ESP32-S3-CAM the ESP32 board (component `U2`, footprint `FujiNet:ESP32-S3-CAM`). It runs the FujiNet firmware, holds the WiFi radio, and carries the microSD slot.

Universal prototype board `fujiversal-pcb-prototype/Bus-proto` (`Universal-proto-v1`). It seats both dev boards, distributes power, and routes every RP2350 GPIO to a generic **bus header** through a farm of solder jumpers. That header is an 8-bit ISA card-edge — see the next section for why.

Bus adapter a small per-machine board that converts the universal board's bus header into the target machine's physical connector. The repository already contains `CoCo-adapter` and `MSX-adapter`; building the **ISA adapter** is Chapter 6.

> **NOTE**
> The Freenove board may need a hardware tweak before its microSD works: one SD pin must be grounded by a solder bridge. The prototype README flags this (`<insert details about soldering microSD pin to ground here>`); confirm against your board revision before assembly.

2.2 Why the inter-board bus is shaped like ISA

When the prototype board was designed, its generic bus header was given the footprint of an 8-bit ISA edge connector (`Connector:Bus_ISA_8bit`, footprint `parts:ISA_8bit`). Both the CoCo and MSX adapters carry a matching `Bus_ISA_8bit` connector on the side that plugs into the universal board, and the machine's real connector (`CoCo-edge`, `MSX-Edge`) on the other.

This has a happy consequence for us: **for an ISA bring-up the adapter is nearly a pass-through**. The universal board's GPIO map was drawn directly from the ISA signal list — `GP0` is `A0`, `GP20` is `D0`, `GP28` is `/MEMR`, and so on (the full map is Table 9). The ISA adapter's job is therefore not signal translation but **electrical conditioning**: getting the 5-volt ISA bus safely in and out of a 3.3-volt RP2350, and presenting gold fingers that fit a real PC slot.

2.3 The two firmware images and one ROM image

Three pieces of code run in a working system:

`fujiversal` UF2	RP2350	`fujiversal/` — `make ROM_FILE=… BOARD=…`
FujiNet firmware	ESP32-S3	`fujinet-firmware/` — PlatformIO env
Host ROM	host CPU (served by RP2350)	`fujinet-config/` — per-platform loader + CONFIG

Table 2. The three build products. The host ROM is **data** compiled into the RP2350 image (`build/<board>/rom.h`), not a separate flash.

The host ROM deserves emphasis because it is easy to forget. The RP2350 does not invent the bytes the host CPU fetches from the emulated ROM; those bytes come from `fujinet-config` , which builds a small loader and the CONFIG user interface for each platform. You build that ROM image **first**, then build `fujiversal` with `ROM_FILE` pointing at it (Chapter 15).

2.4 The communication layers

From the host CPU down to the internet, a single disk read passes through five distinct layers. Knowing which layer owns a problem is the difference between an hour and a week of debugging.

	Layer	Owner	Responsibility
5	**Device / media**	ESP32	Fuji, disk, N:, printer, clock; image formats; N: protocol adapters
4	**FujiBus (FEP-004)**	ESP32 ⇄ host lib	device + command + AUX fields + payload, SLIP-framed, checksummed
3	**Byte pipe**	RP2350 ⇄ host	the 4 I/O registers: GETC / STATUS / PUTC / CONTROL
2	**Bus interface (PIO)**	RP2350	ROM emulation, address decode, drive/sample the data bus
1	**Physical bus**	adapter + board	voltage levels, connector, timing, AEN / strobes

Figure 2. The five layers of a FujiNet transaction on a tandem platform. The chapters of this guide are organised bottom-up: Part II builds layer 1, Part III builds layers 2–3, Part IV builds layers 4–5.

CHAPTER 3

The FujiBus protocol (FEP-004)

FujiBus is the packet protocol the ESP32 and the host exchange. It is the working implementation of the FEP-004 proposal, and it is identical whether the bytes travel over a true RS-232 line, an MSX serial port, or — in our case — the USB-CDC link between the RP2350 and the ESP32. This chapter is a precise reference; you will implement an encoder for it in the client library (Chapter 16) and decode it in the PIO bridge's control path (Chapter 8).

The authoritative implementations are `FujiBusPacket.cpp` (present in both `fujiversal/` and `fujinet-firmware/lib/bus/rs232/`) and the C encoder in `fujinet-lib-experimental`. Where the FEP-004 wiki draft left questions open, the live code answers them; this chapter documents the code.

3.1 Framing: SLIP

Every packet is wrapped in SLIP (RFC 1055) so the receiver can find frame boundaries in a raw byte stream. A frame **both starts and ends** with the `END` byte.

`END`	`0xC0`	Frame delimiter (sent before and after every frame)
`ESC`	`0xDB`	Escape prefix
`ESC_END`	`0xDC`	Follows `ESC` to mean a literal `0xC0`
`ESC_ESC`	`0xDD`	Follows `ESC` to mean a literal `0xDB`

Table 3. SLIP framing bytes (`FujiBusPacket.h`). Any `0xC0` in the payload is sent as `0xDB 0xDC`; any `0xDB` as `0xDB 0xDD`.

3.2 The packet header

Inside the SLIP frame is a fixed six-byte header followed by optional field descriptors, optional AUX fields, and an optional payload. All multi-byte values are **little-endian**.

device 1	command 1	length 2 (LE)	checksum 1	descr 1	fields… AUX	payload…

0	`device`	Destination device ID (see device table below)
1	`command`	Command ID; in a reply, `ACK` (`0x06`) or `NAK` (`0x15`)
2–3	`length`	Total decoded packet length **including** the header, little-endian
4	`checksum`	8-bit add-with-carry-fold over the whole packet with this byte zeroed
5	`descr`	First field descriptor (see below); `0x00` if there are no AUX fields

Table 4. The six-byte FujiBus header (`struct fujibus_header`, `FujiBusPacket.cpp`).

> **NOTE**
>
> The header struct is declared `static_assert(sizeof(fujibus_header) == 6)` and the checksum byte is asserted to be at offset 4. If you re-implement the header in another language, preserve this layout exactly — the firmware reads it as a packed C struct.

3.3 The checksum

The checksum is **not** a plain XOR. It is an 8-bit running sum that folds the carry back in after every byte, computed over the entire packet (header + descriptors + AUX + payload) with the checksum byte set to zero:

```
uint8_t calcChecksum(const ByteBuffer &buf) {
    uint16_t chk = 0;
    for (size_t i = 0; i < buf.size(); ++i) {
        chk += buf[i];
        chk = (chk >> 8) + (chk & 0xFF);   // fold carry
    }
    return (uint8_t) chk;
}
```

3.4 Field descriptors and AUX

Most FujiNet commands carry a few small "AUX" arguments — a disk sector number, a network open mode, a string length. FEP-004 packs these compactly. The `descr` byte (and any additional descriptor bytes) encodes how many AUX values follow and how wide each is.

`descr`			`FUJI_FIELD_*`
0	0	—	`NONE`
1	1	1 byte	`A1`
2	2	1 byte	`A1_A2`
3	3	1 byte	`A1_A2_A3`
4	4	1 byte	`A1_A2_A3_A4`
5	1	2 bytes	`B12` (a `uint16`)
6	2	2 bytes	`B12_B34` (two `uint16`)
7	1	4 bytes	`C1234` (a `uint32`)

Table 5. Field-descriptor encoding. The two lookup tables in the code are `numFieldsTable = {0,1,2,3,4,1,2,1}` and `fieldSizeTable = {0,1,1,1,1,2,2,4}`.

Bit 7 of a descriptor (`FUJI_DESCR_ADDTL_MASK`, `0x80`) means "another descriptor byte follows", letting a packet mix field widths (for example a 32-bit sector number **and** a one-byte unit). AUX values are written little-endian immediately after all descriptor bytes; anything left over is the payload.

> **NOTE**
> You rarely hand-assemble descriptors. The client library exposes `fuji_bus_call(device, cmd, fields, a1, a2, a3, a4, data, len, reply, reply_len)` plus a family of `FUJICALL_*` macros (`FUJICALL_A1`, `FUJICALL_B12_D`, `FUJICALL_C1234`, …) named for exactly these field codes. Chapter 16 shows the byte pipe underneath them.

3.5 Devices and commands

A packet's first byte selects a device. The RP2350 also watches this byte: a packet addressed to device `0xFF` (`FUJI_DEVICEID_DBC`, the bus controller itself) is consumed by the RP2350 and never reaches the ESP32 (Chapter 8).

`0x31` – `0x3F`	`DISK` … `DISK_LAST`	Virtual disk drives (block devices)
`0x40` – `0x43`	`PRINTER` … `PRINTER_LAST`	Printers / voice
`0x45`	`CLOCK`	Real-time clock (APETime)
`0x50` – `0x53`	`SERIAL`	Serial / modem passthrough
`0x5A`	`CPM`	CP/M console
`0x70`	`FUJINET`	The Fuji control device (mounts, hosts, config)
`0x71` – `0x78`	`NETWORK` … `NETWORK_LAST`	`N:` network units (8 of them)
`0x99`	`MIDI`	MIDI
`0xFF`	`DBC`	Bus controller (RP2350) — intercepted locally

Table 6. FujiNet device IDs (`fujiDeviceID.h`). The same header is shared, byte-for-byte, by the RP2350 firmware and the client library.

Commands are a single byte. Many are printable ASCII, a convention inherited from the Atari SIO and `N:` heritage: `'O'` (`0x4F`) opens, `'R'` (`0x52`) reads, `'W'` (`0x57`) writes, `'S'` (`0x53`) gets status, `'C'` (`0x43`) closes. The Fuji control device uses a dense high range (`0xD0` – `0xFF`) for mounts, host slots, app-keys, hashing, and the rest. The complete list is `fujiCommandID.h`, reproduced in Appendix A.

3.6 Request and response

The exchange is strictly command/response over the byte pipe:

1. The host (via the client library) builds a packet, SLIP-encodes it, and streams it out through the `PUTC` register.
2. The ESP32 decodes it, dispatches to the addressed device, and acts.
3. The ESP32 replies with a packet whose `command` is `ACK` (`0x06`) on success — carrying any requested data as payload — or `NAK` (`0x15`) on failure.
4. The host polls `STATUS` for the `available` bit, then reads the reply bytes back through `GETC` .

> **NOTE**
> The FEP-004 wiki draft explicitly left “reply packet structure” and “how to signal data availability” as open questions. The shipping code resolves them: replies are **full FujiBus packets** (same header, checksum, optional payload), and availability is signalled by the `STATUS` register’s `available` bit in the byte pipe — no platform-specific interrupt line required. Document your platform’s behaviour the same way.

PART II

Hardware Bring-Up

The physical layer: enough ISA to decode it correctly, the anatomy of the universal prototype board, how to build the ISA adapter, and the jumper-by-jumper checklist that gets you to first power-on without releasing the magic smoke.

CHAPTER 4

The ISA bus in one sitting

You do not need to be an IBM PC historian to bring FujiNet up on ISA, but you must decode the bus **exactly**, because a card that mis-decodes an I/O or memory cycle will either do nothing or corrupt the machine. This chapter is the minimum correct model of the 8-bit (PC/XT) ISA bus. If you are porting to a different bus, this is the chapter you replace.

4.1 Two address spaces, four strobes

The 8088 has **separate** I/O and memory address spaces, and the ISA bus exposes both with separate read and write strobes. A card decides what kind of cycle is happening by watching which strobe falls.

`/MEMR`	low	CPU reads memory — the card must drive `D0–D7` if the address is its ROM
`/MEMW`	low	CPU writes memory
`/IOR`	low	CPU reads an I/O port — drive `D0–D7` if the port is ours
`/IOW`	low	CPU writes an I/O port — latch `D0–D7` if the port is ours

Table 7. The four 8-bit ISA command strobes. On the universal board these are `GP28` – `GP31`. A FujiNet card uses `/MEMR` for its boot ROM and `/IOR` / `/IOW` for its byte-pipe registers.

4.2 The signals that matter, and AEN above all

`A0–A19`	in	20-bit memory address. I/O ports decode only `A0–A9`.
`D0–D7`	bi	8-bit data. You drive it only during **your** read cycles.
`AEN`	in	**Address Enable.** High during DMA, when the addresses on the bus are **not** a CPU I/O cycle. An I/O card MUST qualify its decode with `AEN` low, or it will respond to DMA addresses and crash the machine.
`ALE` / `BALE`	in	Address Latch Enable. The address is valid to latch on its falling edge.
`RESET DRV`	in	Active-high system reset. Use it to reset your state machines.
`CLK`	in	System bus clock, about 4.77 MHz on a PC/XT.
`OSC`	in	14.318 MHz oscillator (rarely needed by a peripheral).
`I/O CH RDY`	out (o.c.)	Wait-state line. Pull it low to stretch a cycle until your data is ready.
`IRQ2–IRQ7`	out	Interrupts. Optional for FujiNet (the byte pipe is polled).
`DRQ / DACK`	bi	DMA handshake. Not used by FujiNet.

Table 8. The 8-bit ISA signals relevant to a FujiNet card, and how the universal board labels them. The full 62-pin pinout is Appendix B.

> **IMPORTANT**
> `AEN` is the single easiest signal to get wrong and the most destructive. The decode condition for **any** I/O port on ISA is "address matches **and** `AEN` is low". The universal board routes `AEN` to `GP32` precisely so your PIO program can gate on it. Forgetting it is the classic first-bring-up bug.

4.3 Decoding our two windows

A FujiNet ISA card presents two address windows to the host:

A boot ROM (optional but recommended) a block in the **expansion ROM** region, `0xC0000` – `0xDFFFF`. The PC BIOS scans this region in 2 KB steps at power-on looking for option ROMs, each marked by the signature bytes `0x55 0xAA`, a length byte, and an entry point. A FujiNet boot ROM lets the machine boot straight from a mounted disk image. For the worked example we place it at `0xC8000`.

An I/O window of four ports the byte pipe. Any free range works; the prototype/experiment range `0x300` – `0x31F` is the conventional choice for a home-brew card, so we use `0x300` – `0x303` for `GETC`, `STATUS`, `PUTC`, and `CONTROL`. (Compare the MSX build, which memory-maps the same four registers at `0xBFFC`, and the CoCo build at `0xFF41` — the byte pipe is identical; only the decode address changes.)

> **NOTE**
> The ROM window is decoded against `A0–A19` with `/MEMR`. The I/O window is decoded against `A0–A9` with `/IOR` / `/IOW` **and** `AEN` low. Two windows, two decode rules — both implemented in the PIO `wait_sel` program in Chapter 9.

4.4 Timing, and why FujiNet hides behind a poll

A PC/XT bus read gives a card on the order of a few hundred nanoseconds to put data on `D0–D7` after the strobe falls. The RP2350 PIO can meet that for a value it **already has** — a ROM byte, or whatever is sitting in the byte-pipe FIFO. What it cannot do is fetch a fresh byte from the ESP32 (USB + WiFi latency is measured in milliseconds) inside one bus cycle.

The byte pipe is designed around this gap. The host never blocks the bus waiting for the network. It reads `STATUS`, and only when the `available` bit is set does it read `GETC`, which returns a byte the RP2350 already buffered. The slow path — RP2350 to ESP32 to the internet and back — runs entirely between bus cycles. This is why a FujiNet card does not normally need to assert `I/O CH RDY`: there is no long wait to insert.

> **TIP**
> Keep `I/O CH RDY` available on the adapter anyway. If you ever extend the design to a synchronous register (one whose value must be computed during the read), you will need it, and adding it later means cutting traces.

CHAPTER 5

Anatomy of the universal prototype board

The board that does the work is `fujiversal-pcb-prototype/Bus-proto` (`Universal-proto-v1`). This chapter is its field guide: what each connector is, how power flows, and — the part you will actually touch — the solder-jumper farm and the test points.

5.1 The GPIO-to-ISA map

The heart of the board is the fixed mapping from RP2350 GPIO to bus signal. It was drawn from the ISA signal list, which is why ISA is the native case. Memorise the shape of it: address low, data middle, control high.

GP0	A0	GP8	A8	GP16	A16
GP1	A1	GP9	A9	GP17	A17
GP2	A2	GP10	A10	GP18	A18
GP3	A3	GP11	A11	GP19	A19
GP4	A4	GP12	A12	GP20	D0
GP5	A5	GP13	A13	GP21	D1
GP6	A6	GP14	A14	GP22	D2
GP7	A7	GP15	A15	GP23	D3
GP24	D4	GP28	/MEMR	GP32	AEN
GP25	D5	GP29	/MEMW	GP33	CLK
GP26	D6	GP30	/IOR	GP34	ALE
GP27	D7	GP31	/IOW	GP35	RESET

Table 9. The Universal-proto-v1 GPIO map (net labels `GP0_ISA_A0` … `GP35_ISA_RESET` in the schematic). `GP36` – `GP47` are unassigned spares brought out to the breakout headers.

> **NOTE**
> This map is also your PIO pin-define table. In Chapter 9 these exact numbers become the `.define A0_PIN 0` , `.define D0_PIN 20` , `.define IOR_PIN 30` lines of `boards/isa_proto.pio` . Keeping the PIO defines numerically identical to this table is what makes the hardware and firmware agree.

5.2 Connectors and headers

J1	Bus_ISA_8bit edge	The universal bus header. The ISA adapter (or CoCo/MSX adapter) mates here.
J2	ISA Breakout	Every ISA-side signal on 0.1″ pins — your logic-analyzer tap.
J3 / J4	Pico GPIO Breakout 1 & 2	Every RP2350 GPIO on 0.1″ pins — probe the chip side here.
J5 – J8	Conn_01x20	The seats for the Core2350B and ESP32-S3 modules.
J9	Conn_02x02	Power-source selection (see below).
U1	WaveShare Core2350B	The RP2350B module.
U2	ESP32-S3-CAM	The ESP32 module.
D1	diode	Power-rail protection.

Table 10. Connectors and major parts on Universal-proto-v1. There are **no buffer ICs**: every GPIO reaches the bus directly through a solder jumper.

⚠ MAGIC SMOKE — HARDWARE HAZARD

Read tbl-connectors again: U1 and U2 are the only active parts. The RP2350's GPIO connect to the 5-volt ISA bus **directly**, through solder jumpers, with no level shifting. RP2350 GPIO are not 5-volt tolerant. The prototype gets away with it for careful, powered-down, one-signal-at-a-time bring-up; a card left in a running PC this way is living on borrowed time. Chapter 6 covers the buffering you add to make it safe. The board even ships with a MagicSmoke.svg — take the hint.

5.3 The solder-jumper farm

Thirty-nine solder jumpers sit between the GPIO map and the bus. They are your isolation and configuration controls.

JP1 – JP36	bridged (SolderJumper_2_Bridged)	One per bus signal. Each connects a single GPIO to its ISA net. **Cut** one to lift that signal — to isolate it, to splice a buffer in series, or to free the GPIO for another use.
JP37 – JP39	open (SolderJumper_2_Open)	Optional straps — alternate routing / configuration. **Bridge** one only when a specific option calls for it.

Table 11. The solder-jumper farm. Bridged-by-default jumpers are isolation points; open-by-default jumpers are options.

The bring-up discipline these enable is simple and worth stating: **you can verify the board one signal at a time**. Cut every jumper, bring the RP2350 up with no bus connection, then bridge signals back in groups (power and ground first, then address, then strobes, then data) checking each with the breakout headers as you go.

5.4 Power

The board can take power from the ISA bus (+5V, with ±12V and -5V also present on the edge), from the dev boards' USB, or from a bench supply on the header. J9 selects the source and D1 protects against back-feed. The ESP32 rails appear in the schematic as E_5V / E_3v3.

CAUTION

Decide your power source **before** you plug into a PC. If both USB and the ISA +5V are live and the board is not strapped for it, you are tying two supplies together. During bench bring-up, power from USB and leave the ISA +5V jumper open; only switch to bus power once the card is otherwise proven.

CHAPTER 6

Building the ISA adapter

The adapter is the one new PCB this platform needs. Because the universal board's bus header is already an 8-bit ISA edge, the ISA adapter is the simplest of the family — but "simplest" is not "trivial", because this is where the 5-volt problem gets solved.

6.1 What the adapter is

Look at the two adapters already in the repo for the pattern. Each has a `Bus_ISA_8bit` connector that mates the universal board, and a machine-specific edge on the other side:

`CoCo-adapter`	`CoCo-edge` (`P1`)	`Bus_ISA_8bit`
`MSX-adapter`	`MSX-Edge` (`J1`)	`Bus_ISA_8bit`
ISA-adapter (you build it)	real ISA gold fingers (`parts:ISA_8bit`)	`Bus_ISA_8bit`

Table 12. The adapter family. The CoCo and MSX adapters re-map signals; the ISA adapter mostly conditions them, because the two edges are the same shape.

In the CoCo and MSX cases the adapter is mostly a **wiring** problem: the 6809 or Z80 pinout must be mapped onto the universal board's ISA-shaped header (the adapters' net labels — `A0-A15` , `/CTS` , `/SCS` , `E` for CoCo; `/SLTSL` , `/MREQ` , `/IORQ` , `/RD` , `/WR` for MSX — show that translation). For ISA, the signals already line up one-to-one, so the adapter's job is electrical.

6.2 The card edge

Use the `parts:ISA_8bit` footprint (the schematic symbol is `Connector:Bus_ISA_8bit` ; its `descr` is "AT ISA 16 bits Bus Edge Connector" but the 8-bit 62-pin subset is what we populate). Mind two mechanical details:

1. **Orientation.** The gold fingers must seat in a real slot, so the component side and the key notch follow the ISA mechanical spec — get this mirrored and the card is electrically backwards.
2. **Thickness and gold.** A card edge that plugs into a motherboard slot wants 1.6 mm FR-4 and hard-gold fingers; ENIG will wear. For early bring-up a slot **extender** or a socketed breakout saves the fingers.

6.3 Solving the 5-volt problem

This is the real content of the adapter. Three approaches, in increasing order of safety and effort:

Direct (as prototyped)	GPIO straight to the bus through the jumpers. Inputs see 5 V on a non-5-V-tolerant pin.	Bench only, powered-down probing
Series resistors + clamps	330 Ω in each bus line; the RP2350's ESD diodes clamp, the resistor limits current. Crude but cheap.	Short-lived prototypes
Buffered (recommended)	`74LVC245` octal transceivers on the data bus and `74LVC` buffers on the address/strobe inputs. 5-V-tolerant `LVC` inputs accept the bus; outputs drive 3.3 V to the RP2350.	Production

Table 13. Three ways to bridge 5 V ISA and 3.3 V RP2350. Build the adapter for the third even if you start on the first.

The buffered design needs one piece of logic the simpler buses do not: **the data transceiver's direction must be driven**. `D0–D7` are inputs to the card except during a host read of **our** address, when the card drives them. Generate the `74LVC245` direction/enable from the decode:

```
DIR(A->B, card drives bus) = our_read_cycle
  our_read_cycle = (/IOR low AND AEN low AND port in 0x300..0x303)
                OR (/MEMR low AND addr in ROM window 0xC8000..)
OE# (enable)     = our_cycle  (read OR write of one of our windows)
```

You can produce these terms with a small GAL/ATF16V8 or a couple of `74LVC` gates on the adapter, **or** let the RP2350 itself drive the `245` direction from a spare GPIO (`GP36` – `GP39`) — the PIO already computes "we are selected and this is a read". Driving the buffer from the PIO keeps the adapter purely passive, at the cost of one GPIO and tight timing on the direction signal.

> **NOTE**
> The MSX and CoCo bring-ups dodge most of this because their cartridge edges are closer to the RP2350's comfort zone and their `read` PIO programs flip pin directions in step with the system clock (the CoCo `read.pio` side-sets pin direction around the 6809 `E` clock). On ISA there is no convenient single clock edge to hang the data-direction flip on — you decode it from the strobes. Plan the buffer direction logic early; it is the adapter's only real complexity.

6.4 Power and decoupling

Bring `+5V` and `GND` (and `±12V` / `−5V` if you want them on the breakout) from the ISA edge to the universal board's power header, gated by the `J9` selection discussed above. Put a bulk capacitor (10 µF) and a 0.1 µF per buffer IC right at the adapter; ISA backplane power is noisy and the card edge is a long way from the PC's regulator.

CHAPTER 7

Hardware configuration and first power-on

Everything in Part II comes together here as a procedure. Follow it in order; do not skip the powered-down continuity checks.

7.1 ISA jumper and strap settings

`JP1` – `JP36`	bridged for `A0–A19`, `D0–D7`, `/MEMR`, `/MEMW`, `/IOR`, `/IOW`, `AEN`, `RESET`; the rest cut	Connect only the signals the ISA PIO uses; leave `CLK` / `ALE` jumpers cut unless your decode needs them.
`JP37` – `JP39`	open	No optional strap is required for the baseline ISA build.
`J9` power	USB during bench bring-up; bus `+5V` only when deployed	Avoid tying USB and ISA `+5V` together (Chapter 5).
microSD mod	apply per board revision	The Freenove SD pin-to-ground bridge (Chapter 2).

Table 14. Baseline jumper configuration for the ISA worked example.

7.2 Test points and probing

The breakout headers are the whole point of the prototype board. Wire a logic analyzer to `J2` (ISA side) and `J3` / `J4` (GPIO side) and you can watch a bus cycle cross the board.

`J2` `AEN`, `/IOR`, `A0–A9`	Is the host actually addressing our port, and is `AEN` low when it does?
`J3` / `J4` `GP20–GP27`	Is the RP2350 driving `D0–D7` at the right instant, and releasing them after?
`J3` / `J4` `GP30` / `GP31`	Does the PIO see `/IOR` / `/IOW` cleanly, or is there ringing from the unbuffered edge?
`J2` `I/O CH RDY`	Is anything stretching the cycle unexpectedly?

Table 15. A starting probe map. Trigger the analyzer on `/IOR` falling with `AEN` low to capture exactly our read cycles.

7.3 The power-on sequence

1. **Continuity, unpowered.** With an ohmmeter, confirm `+5V` is not shorted to `GND` and that each bridged jumper connects the GPIO you expect to the ISA pin you expect. Cut jumpers stay cut.
2. **RP2350 alone.** Power the Core2350B from USB only, no bus connection. Confirm it enumerates as a USB CDC device (Chapter 10). Nothing should get warm.
3. **ESP32 alone.** Power and flash the ESP32; confirm WiFi and SD (Chapter 11). Still no bus.
4. **Bus, address only.** Power the rig on the bench, bridge the address and `AEN` jumpers, and use the breakout to watch the PIO latch addresses as you (or a test program) touch the I/O window. Data jumpers still cut.
5. **Bus, full.** Bridge `/IOR` / `/IOW` / `/MEMR` and the data jumpers. Run the loopback test of Chapter 10.
6. **In the machine.** Only now, with the adapter buffered, move the card into a real slot and switch `J9` to bus power.

> **TIP**
> Keep a powered USB hub between your workstation and both dev boards. You will reflash the RP2350 and ESP32 many times during bring-up, and a hub with per-port power lets you cycle one board without disturbing the other or the bus.

PART III

The RP2350 Bus Interface

The `fujiversal` firmware: how two cores and three PIO state machines turn a generic dev board into a ROM-and-registers peripheral, and how to write the PIO program that decodes **your** bus. This is the most platform-specific code you will write.

CHAPTER 8

Inside the fujiversal firmware

`fujiversal` is a single Pico-SDK application that emulates a ROM chip and a four-register I/O window on the host's bus, and bridges that window to the ESP32 over USB. Before you write a line of ISA PIO, understand how the existing MSX and CoCo builds are structured, because your build is the same skeleton with a new `.pio` file.

8.1 Two cores

The firmware pins the time-critical work to one core and the housekeeping to the other.

Core 1 — `romulan()` the bus loop. It runs `__time_critical_func`, sets up the PIO, and then spins forever pulling latched bus words from a PIO FIFO and responding to them. It is the only code allowed to touch the bus. It services ROM reads and the I/O registers with no operating system, no allocation, and no blocking.

Core 0 — `main()` the USB bridge. It runs TinyUSB, moves bytes between the host (via core 1) and the ESP32 (via USB-CDC), maintains the SLIP framing, and intercepts the handful of packets addressed to the bus controller itself. It also feeds the watchdog.

The two cores exchange single bytes through the SDK's multicore FIFO, buffered on each side by a 1 KB ring.

8.2 The three PIO state machines

A bus-based board uses three PIO programs, set up in `setup_pio_irq_logic()`. They are defined per board in `boards/<board>.pio` and are the part you rewrite for a new bus.

`wait_sel`	Watch the select/decode signals. When the host addresses the card, raise an IRQ. This program **encodes your bus's decode rule**.
`send_bus`	On that IRQ, sample **all 32 low GPIO** in one instruction and autopush the 32-bit word to the RX FIFO. Core 1 reads it as a `BusSignals` union.
`read`	Drive `D0-D7` with a byte core 1 supplies, manage the data-pin direction (high-Z except during a read of our address), and release the bus when the cycle ends.

Table 16. The three PIO state machines. `wait_sel` and `read` are where bus timing lives; `send_bus` is essentially boilerplate.

8.3 The BusSignals union

`send_bus` hands core 1 a packed 32-bit word. A C union maps the bit positions of the GPIO onto named fields. This is the MSX definition; note how `addr:16`, `data:8`, and the control bits line up with the pin `.define`s in the same file:

```
typedef union {
  struct {
    uint32_t addr:16;     // A0..A15  on GP0..GP15
    uint32_t resv:4;
    uint32_t data:8;      // D0..D7   on GP20..GP27
    uint32_t rd:1;        // /RD
    uint32_t wr:1;        // /WR
    uint32_t iorq:1;      // /IORQ
    uint32_t memrq:1;     // /MEMRQ
  } __attribute__((packed));
```

```
  uint32_t combined;
} __attribute__((packed)) BusSignals;
```

Your ISA union (Chapter 9) is wider in the address field and carries different control bits, but the idea is identical: the bit layout **is** the GPIO map.

8.4 ROM emulation and the I/O window

Core 1's loop is short enough to read in full. It decides, for every latched bus cycle, whether the address falls in the I/O window or the ROM window, and acts:

```
// from romulan(), main.cpp — the per-cycle decode
if (IO_BASE <= bus.addr && bus.addr < IO_TOP) {
    unsigned io_reg = (bus.addr - IO_BASE) & 0x3;
#ifdef RW_PIN
    if (!bus.rw) io_reg |= 2;          // fold R/W into the register index
#endif
    switch (io_reg) {
    case IO_GETC:   pio_put_fifo(PSM_READ, sio_hw->fifo_rd); break;   // byte from
ESP32
    case IO_STATUS: pio_put_fifo(PSM_READ,
                        sio_hw->fifo_st & SIO_FIFO_ST_VLD_BITS
                          ? IO_FLAG_AVAIL : 0x00); break;              // is a byte
ready?
    case IO_PUTC:   sio_hw->fifo_wr = bus.combined; break;             // byte to ESP32
    case IO_CONTROL: break;
    }
}
else if (BUS_ROM_BASE <= bus.addr && bus.addr < BUS_ROM_TOP) {
    rom_offset = bus.addr - BUS_ROM_BASE;
    pio_put_fifo(PSM_READ, rom_ptr[rom_offset]);                       // serve a ROM
byte
}
```

Two things to take from this:

1. The four registers are nothing more than the two ends of the multicore FIFO (`sio_hw->fifo_rd` / `fifo_wr`) dressed up as bus-visible ports. `GETC` pops a byte the ESP32 sent; `PUTC` pushes a byte toward the ESP32; `STATUS` exposes the FIFO's "valid" flag as the `available` bit.
2. `IO_BASE` , `IO_GETC` , `IO_STATUS` , `IO_PUTC` , `IO_CONTROL` , `IO_FLAG_AVAIL` , `BUS_ROM_BASE` , and `BUS_ROM_TOP` are all `.define` s in the board's `.pio` file. Re-pointing the byte pipe at ISA's address map is mostly a matter of changing those constants.

8.5 ROM bank-switching: the one packet the RP2350 keeps

There is a single exception to "the RP2350 is a transparent pipe". The host-side loader needs to swap the emulated ROM's contents at runtime (to page in CONFIG, or a booted disk's loader). It does this with FujiBus packets addressed to `FUJI_DEVICEID_DBC` (`0xFF`). Core 0 sniffs the second byte of every frame and, if it is `0xFF` , consumes the packet locally instead of forwarding it:

```
// main loop, core 0 — divert DBC frames to process_command()
if (command_size == 2 && input != FUJI_DEVICEID_DBC) {
    // second byte isn't 0xFF -> not for us, push to ESP32
    ...
}
```

```
else if (command_size > 1 && input == SLIP_END) {
    process_command(command_buf);     // a complete DBC frame: handle locally
}
```

`process_command()` implements a tiny device: `FUJICMD_OPEN` selects a RAM bank, `FUJICMD_WRITE` fills it, `FUJICMD_CLOSE` activates it (the next ROM fetch sees the new image), and `FUJICMD_RESET` reverts to the built-in ROM. You inherit this for free; an ISA build needs no change here.

8.6 USB transport and SLIP

Everything else core 0 does is plumbing: read a byte from USB (`tud_cdc_read`), and if a frame is in progress (started by a `SLIP_END`) accumulate it, otherwise pass the byte straight to core 1 for the host to read via `GETC`. Bytes the host writes via `PUTC` travel the other way, out through `tud_cdc_write`. A 50 ms timeout flushes a partial frame so a glitch cannot wedge the pipe. The USB side is `stdio`-based on some builds and raw TinyUSB CDC on others; either way the ESP32 sees a clean CDC-ACM serial port.

CHAPTER 9

Writing the ISA PIO program

This chapter builds `boards/isa_proto.pio` — the file that teaches `fujiversal` to decode ISA. The repository ships `msx_proto_260402.pio` and `coco_proto_260402.pio`; an ISA file does not yet exist, so this is genuinely new firmware. Use the MSX file as your structural template (it, like ISA, has separate I/O and memory strobes), and change three things: the pin defines, the decode in `wait_sel`, and the data-direction timing in `read`.

> **IMPORTANT**
> The PIO listings in this chapter are a **worked starting point** transcribed in the style of the shipping `.pio` files, not a build the repository has yet proven on silicon. Treat them as a design to bring up with a logic analyzer (Chapter 10), not as drop-in code. Every value in them traces to the GPIO map (`tbl-gpmap`) and the ISA decode rules (Chapter 4).

9.1 Pin defines and constants

Start from the GPIO map. These defines must match `tbl-gpmap` exactly, because the same numbers index the PIO's `wait`, `mov pins`, and `jmp pin` instructions.

```
.pio_version 1

.define public DATA_WIDTH   8
.define public ADDR_WIDTH   20          ; ISA carries A0..A19

.define public A0_PIN       0           ; A0..A19  -> GP0..GP19
.define public D0_PIN       20          ; D0..D7   -> GP20..GP27
.define public MEMR_PIN     28          ; /MEMR
.define public MEMW_PIN     29          ; /MEMW
.define public IOR_PIN      30          ; /IOR
.define public IOW_PIN      31          ; /IOW
.define public AEN_PIN      32          ; AEN  (HIGH during DMA)
.define public CLK_PIN      33          ; bus CLK
.define public ALE_PIN      34          ; address latch enable
.define public RESET_PIN    35          ; RESET DRV

; --- byte-pipe registers, in ISA I/O space (ports 0x300..0x303) ---
.define public IO_BASE      0x300
.define public IO_GETC      0
.define public IO_STATUS    1
.define public IO_PUTC      2
.define public IO_CONTROL   3
.define public IO_FLAG_AVAIL 0x80

; --- boot ROM window, in the expansion-ROM region ---
.define public BUS_ROM_BASE 0xC8000
.define public BUS_ROM_TOP  0xCC000     ; 16 KB option ROM

.define public IRQ_SEL 0
```

9.2 The BusSignals union for ISA

Because ISA's address is 20 bits, the union differs from the MSX one. Place it in the `decode_addrdata` program block:

```
typedef union {
  struct {
    uint32_t addr:20;      // A0..A19  on GP0..GP19
    uint32_t data:8;       // D0..D7   on GP20..GP27 -- NOTE: starts at bit 20
    uint32_t memr:1;       // /MEMR    GP28
    uint32_t memw:1;       // /MEMW    GP29
    uint32_t ior:1;        // /IOR     GP30
    uint32_t iow:1;        // /IOW     GP31
  } __attribute__((packed));
  uint32_t combined;
} __attribute__((packed)) BusSignals;
```

CAUTION

`send_bus` samples GPIO 0–31, so `AEN` (`GP32`), `CLK` (`GP33`), `ALE` (`GP34`) and `RESET` (`GP35`) are **above** the 32-bit sample window. The baseline design therefore gates `AEN` inside the `wait_sel` program (which can `wait` on any GPIO) rather than reading it in the latched word. If your decode needs `AEN` 's **level** in core 1, move data down to free low bits, or sample with a second `mov` . Getting this boundary wrong is a subtle source of "it responds during DMA" bugs.

9.3 Decode: the wait_sel program

This is the heart of the port. The ISA "we are selected" condition has two forms — an I/O cycle and a memory (ROM) cycle — and an I/O cycle is only valid when `AEN` is low. The program waits for a command strobe to fall while our conditions hold, then raises the select IRQ. Compare the MSX `wait_sel` , which gates on `/SLTSL` ; here we gate on `AEN` and the strobes.

```
.program wait_sel
.wrap_target
idle:
        wait 0 gpio AEN_PIN          ; an I/O cycle requires AEN low
        ; fall through when AEN is low; a memory cycle ignores AEN
        wait 0 gpio IOR_PIN  [1]     ; (illustrative) block until /IOR falls
        irq IRQ_SEL                  ; tell send_bus + core1 we are selected
        wait 1 gpio IOR_PIN          ; hold until the strobe releases
.wrap
```

NOTE

The single-strobe `wait` above is deliberately simplified to show the shape. A complete ISA `wait_sel` must select on `/IOR` **or** `/IOW` **or** `/MEMR` , and pre-qualify the address against the I/O or ROM window so it does not IRQ on every bus cycle in the machine. Two practical ways to do that: (a) decode the high address bits with a few `mov` / `jmp` instructions before the `wait` , or (b) decode the window in external logic (a GAL on the adapter) and feed a single "card selected" line to one GPIO, reducing `wait_sel` to the MSX form. Option (b) is faster to bring up and is recommended for the first board; option (a) removes a part. The CoCo and MSX boards effectively use (b) — the cartridge edge gives them a ready-made select line.

9.4 Driving data: the read program

`read` drives `D0-D7` when core 1 supplies a byte and tri-states them otherwise. The MSX/CoCo versions flip `pindirs` around the system clock; on ISA you flip them around the **strobe**. The shape, using `/IOR` as the read reference and side-set for the direction, mirrors the CoCo `read`:

```
.program read
.side_set 1 opt
        mov x, ~null  side 1         ; X = all ones; start with D0-7 as inputs
.wrap_target
        pull block                   ; wait for core1 to provide the byte
        out pins, DATA_WIDTH         ; place it on D0-D7
        mov osr, ~null
        out pindirs, DATA_WIDTH side 0   ; drive D0-D7 (outputs)
        wait 0 gpio IOR_PIN          ; ... while /IOR (or /MEMR) is low
        wait 1 gpio IOR_PIN          ; host has latched; cycle ending
        mov osr, null
        out pindirs, DATA_WIDTH side 1   ; release D0-D7 back to inputs
.wrap
```

The `send_bus` program needs no ISA-specific change — it samples GPIO and autopushes, exactly as in the MSX file. Keep it verbatim.

9.5 Wiring it into core 1

Core 1's `romulan()` loop already does the right thing for ISA **if** the constants are set, with one addition: ISA distinguishes I/O reads from I/O writes by which strobe is active, so fold the strobe into the register index the way the CoCo build folds `R/W`:

```
if (IO_BASE <= bus.addr && bus.addr < IO_BASE + 4) {
    unsigned io_reg = (bus.addr - IO_BASE) & 0x3;
    if (!bus.iow) {                      // an I/O write cycle
        if (io_reg == IO_PUTC) sio_hw->fifo_wr = bus.data;
        // CONTROL writes (io_reg == IO_CONTROL) handled here if needed
    } else if (!bus.ior) {               // an I/O read cycle
        if (io_reg == IO_GETC)
            pio_put_fifo(PSM_READ, sio_hw->fifo_rd);
        else if (io_reg == IO_STATUS)
            pio_put_fifo(PSM_READ,
                sio_hw->fifo_st & SIO_FIFO_ST_VLD_BITS ? IO_FLAG_AVAIL : 0);
    }
}
else if (!bus.memr && BUS_ROM_BASE <= bus.addr && bus.addr < BUS_ROM_TOP) {
    pio_put_fifo(PSM_READ, rom_ptr[bus.addr - BUS_ROM_BASE]);
}
```

9.6 Adding the board to the build

`fujiversal` selects a board through CMake. Add `isa_proto` alongside the existing boards:

```
# CMakeLists.txt — add isa_proto to the RP2350 board set
if(BOARD STREQUAL "msxrp2350"
   OR BOARD STREQUAL "msx_proto_260402"
   OR BOARD STREQUAL "coco_proto_260402"
   OR BOARD STREQUAL "isa_proto")                # <-- new
    set(PICO_BOARD "pimoroni_pga2350" CACHE STRING "Pico board type" FORCE)
```

```
    set(PICO_PLATFORM "rp2350-arm-s" CACHE STRING "Pico platform" FORCE)
    set(PICO_CHIP "rp2350" CACHE STRING "Pico chip" FORCE)
    # PICO_PIO_USE_GPIO_BASE=1 is required: ISA uses GP0..GP35,
    # which spans past the default 32-pin PIO window.
endif()
```

IMPORTANT
ISA touches GPIO above 31 (`AEN` is `GP32`, `RESET` is `GP35`). The build **must** define `PICO_PIO_USE_GPIO_BASE=1` (the existing RP2350 boards already do) so the PIO can address the upper GPIO bank. The `setup_state_machine()` helper in `setup_sm.cpp` already computes the GPIO base/range and rejects a span that crosses the 16⇔32 boundary illegally — heed its return codes during bring-up.

9.7 Generating the host ROM

The emulated ROM's contents come from `fujinet-config`, not from `fujiversal`. Build the host image for your platform first, then point the `fujiversal` build at it:

```
# 1. build the host-side loader + CONFIG for ISA (in fujinet-config)
#    -> produces an isa rom image, e.g. config-isa.rom
make PLATFORM=isa            # see Chapter 15

# 2. build the RP2350 firmware with that ROM baked in
cd ../fujiversal
make ROM_FILE=../fujinet-config/config-isa.rom BOARD=isa_proto
# -> build/isa_proto/fujiversal_isa_proto.uf2
```

CHAPTER 10

Bringing the RP2350 up

With a `.uf2` in hand, prove the bus interface in isolation before you ask the ESP32 to do anything.

10.1 Flash and enumerate

Hold the BOOTSEL button while plugging the Core2350B into USB; it mounts as a mass-storage drive. Copy the `.uf2` (or use `picotool load`). On reset the board should enumerate as a USB CDC-ACM serial device — that port is the link the ESP32 will later own. Open it from your workstation first; you have a debug console before the ESP32 is in the loop.

10.2 The loopback test

The cleanest first proof needs no host bus at all. With the RP2350 on USB and a terminal open on its CDC port, anything the firmware's core 0 places in the RX ring appears on `GETC`, and anything written to `PUTC` is sent out the CDC port. So:

1. From the workstation terminal, send a byte. Confirm (with a logic analyzer on `J3` / `J4`, or a tiny host test program) that a host read of `GETC` returns it and that `STATUS` showed `available` first.
2. Have the host write a byte to `PUTC`; confirm it arrives on the workstation terminal.

That round trip exercises both PIO directions, the multicore FIFO, and the USB bridge — layers 2 and 3 of `fig-boards` — without any FujiNet firmware running.

10.3 Debugging with the registers

The four registers are also your instrument. A short host program that spins reading `STATUS` and dumping `GETC` is the equivalent of a serial monitor for the byte pipe. When something later goes wrong end-to-end, the question "is the byte pipe healthy?" is answered here, below FujiBus and below the ESP32.

> **TIP**
> Build `fujiversal` with `USE_STDIO` / `VERBOSE_DEBUG` during bring-up to get `printf` over the CDC port, then turn it off: the debug build steals the same CDC channel the ESP32 needs, so a verbose RP2350 and a connected ESP32 cannot coexist. Bring the byte pipe up verbose, then go quiet and hand the port to the ESP32.

PART IV

The ESP32 Device Firmware

Where the work gets easy. On the ESP32 a tandem platform is a FujiBus serial transport, so it reuses the existing `rs232` bus, device, and media classes. This part shows the build target, the pin map, the device and media seams, the host ROM, and the client library.

CHAPTER 11

Where ISA fits in fujinet-firmware

The ESP32 never sees ISA. It sees a CDC-ACM serial port carrying FujiBus packets. That is **exactly** what the firmware's `rs232` bus already consumes — the same class used by the real RS-232 FujiNet and by the MSX serial bring-up. The two `fujiversal` build targets that already exist, `fujiversal-rs232` and `fujiversal-drivewire`, are proof of the pattern: they pair the RP2350 with the ESP32 over USB and select an existing bus on the ESP32 side.

11.1 The rs232 bus is the FujiBus transport

`lib/bus/rs232/` is misleadingly named: it is the FujiBus/FEP-004 engine, not an RS-232 UART driver. It contains its own copy of `FujiBusPacket.cpp` (the same encoder you met in Chapter 3) and a `systemBus` that reads and writes whole packets:

```
// lib/bus/rs232/rs232.h  (abridged)
class systemBus {
    std::forward_list<virtualDevice *> _daisyChain;
    IOChannel *_port;
#if FUJINET_OVER_USB
    ACMChannel _serial;        // USB-CDC host channel  <-- the fujiversal path
#else
    UARTChannel _serial;       // a real UART
#endif
    std::unique_ptr<FujiBusPacket> readBusPacket(int first = -1);
    void writeBusPacket(FujiBusPacket &packet);
    void sendReplyPacket(fujiDeviceID_t source, bool ack,
                         const void *data, size_t length);
    void addDevice(virtualDevice *pDevice, fujiDeviceID_t device_id);
    // ...
};
extern systemBus SYSTEM_BUS;
```

The `FUJINET_OVER_USB` switch is the whole story: when the firmware is built to talk to the RP2350 over USB, `_serial` is an `ACMChannel` (a USB-CDC **host** channel); when it drives a physical serial port, it is a `UARTChannel`. ISA uses the USB path, identical to `fujiversal-rs232`.

11.2 What this means for your effort

> **IMPORTANT**
> For a tandem bus-based platform, you usually write **no new bus class and no new device classes on the ESP32**. You add a build target and a pin map, and reuse `rs232`. New code on the ESP32 is needed only when your platform exposes a device the existing set does not, or needs a disk image format not already handled. Budget your engineering accordingly: the deep work is the PIO (Part III) and the host side (Chapters 15–16), not here.

CHAPTER 12

Adding the platform to the build system

PlatformIO drives the firmware build. Every platform is one `.ini` in `build-platforms/` plus a pin map. Model the ISA target on `platformio-fujiversal-rs232.ini`.

12.1 The build platform file

```
; build-platforms/platformio-fujiversal-isa.ini
[fujinet]
build_bus      = RS232          ; reuse the FujiBus serial bus
build_platform = BUILD_RS232    ; ... and its device/media set

[env:fujiversal-isa]
build_type = debug
build_flags =
    ${env.build_flags}
    -D PINMAP_FUJIVERSAL_ISA            ; <-- new pin map (below)
    -D CONFIG_USB_HOST_ENABLED=1        ; ESP32-S3 is the USB *host*
    -D CONFIG_USB_CDC_ACM_HOST_ENABLED=1
platform         = espressif32@${fujinet.esp32s3_platform_version}
platform_packages = ${fujinet.esp32s3_platform_packages}
board            = esp32-s3-wroom-1-n16r8
```

Three lines carry the design: `build_bus = RS232` chooses the FujiBus transport; `CONFIG_USB_HOST_ENABLED` / `CONFIG_USB_CDC_ACM_HOST_ENABLED` make the ESP32-S3 a USB host so it can open the RP2350's CDC port; and `PINMAP_FUJIVERSAL_ISA` selects your board's pin assignments.

12.2 The pin map

Add a pin-map header guarded by `PINMAP_FUJIVERSAL_ISA` and include it in the firmware's pin-map dispatch. On a `fujiversal` board the pin map is small — the heavy bus I/O lives on the RP2350, so the ESP32 pin map mostly declares the USB-host data pins, the status LEDs, the button, and the SD pins of the Freenove module.

```
// include/pinmap/fujiversal_isa.h   (sketch; mirror fujiversal_rs232.h)
#ifdef PINMAP_FUJIVERSAL_ISA
#define PIN_LED_WIFI     GPIO_NUM_..   // white WiFi LED
#define PIN_LED_BUS      GPIO_NUM_..   // bus activity LED
#define PIN_BUTTON_A     GPIO_NUM_..
// USB host D+/D- and SD pins per the Freenove ESP32-S3-CAM board
// (no parallel-bus pins here: the RP2350 owns the host bus)
#endif
```

> **NOTE**
> The status-LED and button conventions are shared across FujiNet hardware (white = WiFi, an amber/orange = bus, Button A + a safe-reset). Match them so the existing firmware LED/boot logic "just works" — the `fujinet-firmware` `lib/hardware` LED code keys off these names.

12.3 Partitions and flashing

Reuse the existing 16 MB partition layout (`fujinet_partitions_16MB.csv`); the firmware image, the SPIFFS/FAT data partition (web UI, CONFIG assets), and OTA slots are platform-independent. Build and flash with the new env:

```
pio run -e fujiversal-isa
pio run -e fujiversal-isa -t upload    # over the ESP32-S3 USB/JTAG port
```

CHAPTER 13

Device classes

A FujiNet "device" is a class derived from `virtualDevice` that handles FujiBus packets for one device ID. The `rs232` device set you inherit covers the whole standard FujiNet feature list.

13.1 What you inherit

	`lib/device/rs232/`	
`0x70`	`rs232Fuji`	Mounts, host slots, app-keys, hashing, adapter config — the CONFIG back end
`0x31` – `0x3F`	`rs232Disk` + media	Virtual disk drives
`0x71` – `0x78`	`rs232Network`	The `N:` device — one per unit, 8 units
`0x40` – `0x43`	`rs232Printer`	Printer emulation to PDF
`0x50` – `0x53`	`rs232Modem`	Modem / serial passthrough
`0x45`	clock	APETime real-time clock
`0x5A`	`rs232CPM`	CP/M console

Table 17. The device classes a `BUILD_RS232` platform gets for free. Each is a `virtualDevice` registered with `SYSTEM_BUS.addDevice()`.

13.2 The virtualDevice contract

If you ever do need a new device, the interface is small. Every device implements two pure-virtual methods that the bus calls:

```
// lib/bus/rs232/rs232.h
class virtualDevice {
protected:
    fujiDeviceID_t _devnum;
    // Per-command dispatch: usually a switch() on packet.command()
    virtual void rs232_process(FujiBusPacket &packet) = 0;
    // Return 4 status bytes (the historical DVSTAT semantics)
    virtual void rs232_status(FujiStatusReq reqType) = 0;
    virtual void shutdown() {}
public:
    fujiDeviceID_t id() { return _devnum; }
    bool is_config_device = false;   // true for the disk that boots CONFIG
    bool device_active = true;
};
```

The bus reads a packet, finds the device whose `id()` matches `packet.device()`, and calls its `rs232_process()`. To add a device you subclass `virtualDevice`, implement those two methods, and register an instance:

```
SYSTEM_BUS.addDevice(new myDevice(), FUJI_DEVICEID_SOMETHING);
```

> **NOTE**
> Devices are registered during `SYSTEM_BUS.setup()`, which the platform's `BUILD_*` selection wires up. For a stock ISA build you change nothing here — the `rs232` set registers itself.

CHAPTER 14

Media classes

A media class turns a host disk-image file on the SD card into the sectors a virtual disk device serves. They live in `lib/media/`, one directory per platform (`atari`, `apple`, `adam`, `cbm`, `mac`, `drivewire`, `rs232`, …), behind the `MediaType` interface in `lib/media/media.h`.

14.1 Reuse first

The disk device (`rs232Disk`) presents **block** storage: the host reads and writes 512-byte sectors by number (the FujiBus disk command carries the sector as a 32-bit `C1234` field). If your platform's software is happy with raw block images, the existing `rs232` media path already serves them and you add nothing.

14.2 When you need a new media class

Add a `lib/media/<platform>/` only when the host expects an **image format** with structure the firmware must understand — a header, an interleave, a copy-protection scheme, a non-512 sector size. The class implements:

```
// the shape of a MediaType (lib/media/media.h)
class MediaType {
public:
    virtual bool read(uint32_t blockNum, uint16_t *readcount) = 0;
    virtual bool write(uint32_t blockNum, bool verify)        = 0;
    virtual bool format(uint16_t *responsesize)               = 0;
    virtual mediatype_t mount(fnFile *f, uint32_t disksize)   = 0;
    virtual void unmount()                                    = 0;
    static  mediatype_t discover_disktype(const char *filename); // by extension
};
```

For the ISA worked example, decide what the PC boot ROM and DOS expect: a flat sector image (a `.img` of a 360 KB or 720 KB floppy, or a hard-disk image) maps directly onto block reads and needs no new class. A structured format (say a copy-protected original) would. Start flat.

> **TIP**
> `discover_disktype()` dispatches on file extension. When you do add a format, register its extension there so a user mounting `disk.img` from CONFIG gets the right `MediaType` automatically.

CHAPTER 15

The host ROM and CONFIG

The emulated ROM the RP2350 serves is built by `fujinet-config`. This is the code that runs **on the host CPU**, and it is as platform-specific as the PIO — it is written in the host's assembly/C and it drives the byte pipe directly.

15.1 What fujinet-config produces

`fujinet-config` builds, per platform, two things baked into one ROM image:

1. A **loader**: the few hundred bytes the host runs at boot. On ISA this is an option ROM — signature `0x55 0xAA`, a length byte, and an entry point the BIOS calls during its expansion-ROM scan. The loader brings up the byte pipe, asks FujiNet to mount the configured boot disk, and either boots it or launches CONFIG.
2. The **CONFIG application**: the full-screen UI for choosing WiFi hosts, browsing TNFS, and mounting images into the eight disk slots. Its screen text for each platform lives in `fujinet-config/src/<platform>/screen.c` and is rendered in that machine's native character set.

15.2 How the loader talks to FujiNet

The loader uses the same four registers as everything else. In pseudocode, sending one FujiBus byte and reading the reply is:

```
put_byte(b):   while (inb(IO_BASE+IO_STATUS) & BUSY) ;   ; optional flow ctl
               outb(IO_BASE+IO_PUTC, b)
get_byte():    while (!(inb(IO_BASE+IO_STATUS) & IO_FLAG_AVAIL)) ;
               return inb(IO_BASE+IO_GETC)
```

On ISA those `inb` / `outb` are 8088 `IN` / `OUT` instructions to ports `0x300` – `0x303`. On MSX they are memory reads/writes at `0xBFFC`; on CoCo, at `0xFF41`. **Only the access method and the addresses change** — the protocol above is identical, which is exactly why the client library (next chapter) is mostly shared C with a thin per-platform shim.

15.3 Building and chaining the ROM

```
# in fujinet-config
make PLATFORM=isa          # builds loader + CONFIG -> config-isa.rom
```

The output is consumed by `fujiversal` (Chapter 9, `ROM_FILE=…`), which converts it to a C array (`build/<board>/rom.h`) and serves it from `BUS_ROM_BASE`. Change the host UI, rebuild `fujinet-config`, rebuild `fujiversal`, reflash the RP2350.

CHAPTER 16

The client library

`fujinet-lib` is the C library application programmers link against to use FujiNet — the same one CONFIG and every networked program use. The experimental tree, `fujinet-lib-experimental`, is the FujiBus-native version, and its structure is the template for your platform's port.

16.1 The backend pattern

```
fujinet-lib-experimental/
  include/        fujinet-bus.h, fujinet-bus-ezcall.h, FUJI_FIELD_*, FujiDCB
  common/         network.c, network_json.c, fuji_*.c   (platform-independent)
  bus/
    apple2/  atari/  c64/  coco/  msx/  msdos/  adam/    (one dir per platform)
    isa/     <-- you add this
  Makefile        PLATFORMS = coco apple2 atari c64 msx msdos adam   (+ isa)
```

`common/` is the bulk of the library and is shared verbatim. Each `bus/<platform>/` provides the same small surface: the three transport primitives and the platform's port I/O.

16.2 What bus/isa/ contains

`portio.s` / `portio.h`	`inb` / `outb` to the four ports `0x300` – `0x303` (8088 `IN` / `OUT`).
`fujinet-bus-isa.c`	Implements `fuji_bus_call`, `fuji_bus_read`, `fuji_bus_write`: build the FujiBus packet (header, descriptors, AUX, payload, checksum), SLIP-encode it, stream it out `PUTC`, then read the SLIP reply back through `GETC`.

Table 18. The two pieces of a `fujinet-lib` bus backend. Compare `bus/msdos/portio.s` (real PC `IN` / `OUT`) and `bus/msx/portio.s` (memory-mapped) — the ISA backend is closest to the `msdos` one.

The public surface the rest of the library calls is just:

```
// include/fujinet-bus.h
bool   fuji_bus_call(uint8_t device, uint8_t fuji_cmd, uint8_t fields,
                     uint8_t aux1, uint8_t aux2, uint8_t aux3, uint8_t aux4,
                     const void *data, size_t data_length,
                     void *reply, size_t reply_length);
size_t fuji_bus_read (uint8_t device, void *buffer, size_t length);
size_t fuji_bus_write(uint8_t device, const void *buffer, size_t length);
```

and the `FUJICALL_*` convenience macros wrap it with the right `FUJI_FIELD_*` descriptor. A program that opens an `N:` URL never sees a byte of FujiBus framing; it calls `network_open()` in `common/`, which calls `fuji_bus_call()`, which calls your `bus/isa` primitives, which hit the ports.

16.3 Building it

Add `isa` to `PLATFORMS` in the Makefile and build:

```
make isa            # builds fujinet.lib for the isa backend
```

`SRC_DIRS = common bus/%PLATFORM%` already composes the shared core with your new backend; no other Makefile change is needed.

> **TIP**
> Bring the library up against the **loopback** of Chapter 10 before the full firmware is ready: a host program that calls `fuji_bus_write()` and watches the bytes appear on the RP2350's USB console proves your SLIP encoder and port I/O independent of the ESP32. Then connect the ESP32 and the same call returns a real `ACK`.

PART V

Integration & Validation

Putting the layers together: the milestone ladder from first power-on to a booting machine, a troubleshooting matrix indexed by symptom, and what changes when the next platform is not ISA.

CHAPTER 17

The bring-up milestone ladder

Bring a platform up in the order the layers stack, lowest first. Each rung is independently testable, and a failure is contained to the rung you are on. Do not move up until the current rung is solid.

0	Boards seated, power correct	Continuity passes; nothing warms up on USB power; `J9` set for bench.
1	RP2350 enumerates	The Core2350B appears as a USB CDC device on your workstation.
2	ESP32 alive	Firmware flashed (`pio run -e fujiversal-isa -t upload`); WiFi joins; SD mounts.
3	Byte pipe loopback	A host `GETC` returns a byte you injected; a host `PUTC` reaches the USB console (Chapter 10).
4	Address decode	The logic analyzer shows the PIO IRQ firing on **our** I/O cycles and ROM reads, and **not** during DMA (`AEN` high).
5	FujiBus ACK	A `fuji_bus_call()` from the host library returns `ACK` from the ESP32 (the Fuji device answers an adapter-config query).
6	CONFIG boots	The host fetches the option ROM, runs the loader, and CONFIG draws on screen.
7	Mount + boot a disk	CONFIG mounts a `.img` ; the machine boots it through the disk device.
8	`N:` works	A program opens `N:HTTP://…` and reads data over WiFi.

Table 19. The milestone ladder for a tandem bring-up. Milestones 0–4 are hardware/PIO (Parts II–III); 5–8 are firmware/host (Part IV).

> **NOTE**
> Milestones 1–3 need **no host bus at all** — they run on the bench over USB. You can reach milestone 3 before the ISA adapter PCB even arrives. Order your work so the slow-to-fabricate adapter is not on the critical path for the firmware.

CHAPTER 18

Troubleshooting

Indexed by symptom. The “layer” column points you at the part of this guide — and the part of `fig-boards` — that owns the fault.

Machine hangs or reboots the instant the card is inserted	1	I/O decode not gated on `AEN` — the card answers DMA cycles. Confirm `AEN`-low is in the `wait_sel` condition (Ch. 4, 9). Or a 5 V signal is fighting an RP2350 output: check buffering and jumper state.
Card does nothing; no PIO IRQ on the analyzer	2	`wait_sel` decode never matches. Wrong `IO_BASE`, wrong strobe polarity (ISA strobes are active-**low**), or an address jumper cut. Probe `J2` vs `J3` / `J4`.
PIO IRQs fire on every bus cycle	2	`wait_sel` is not pre-qualifying the address window — it selects on the strobe alone. Add the window decode, or feed a decoded select line from adapter logic (Ch. 9).
Host reads our port but gets `0xFF` / garbage	2–3	Data not driven in time, or direction backwards. Check the `read` program's `pindirs` flip timing against the strobe, and the buffer `DIR` term (Ch. 6, 9).
Bytes go out but no reply ever comes back	3–4	Loopback (M3) first. If loopback is fine, the ESP32 isn't opening the CDC port (`CONFIG_USB_CDC_ACM_HOST_ENABLED`?) or the RP2350 is still in verbose `printf` mode stealing the channel (Ch. 10).
`fuji_bus_call` returns failure though bytes flow	4	Framing bug: checksum (the add-with-fold, not XOR), little-endian `length`, or a descriptor/AUX mismatch. Diff your encoder against `FujiBusPacket.cpp` (Ch. 3, 16).
BIOS never runs the option ROM	5	ROM not on a 2 KB boundary, bad `0x55 0xAA`/length/checksum header, or `BUS_ROM_BASE` outside `0xC0000` – `0xDFFFF`. Verify the served bytes at `0xC8000` with the analyzer on `/MEMR` (Ch. 4, 15).
CONFIG draws garbage characters	5	Host-side screen rendering — `fujinet-config/src/<platform>/screen.c` uses the wrong character set or addresses. A firmware/byte-pipe problem would corrupt **data**, not just glyphs.
Intermittent corruption at speed	1	Unbuffered 5 V bus ringing, or missing decoupling at the adapter. This is the design moving off "bench only" — add the `74LVC` buffers (Ch. 6).

Table 20. Troubleshooting matrix. The layer numbers match `fig-boards`; fix the lowest failing layer first.

CHAPTER 19

Porting to a bus that is not ISA

ISA was a forgiving example because the prototype board's pin map was drawn from it. When your next target is a 6502 cartridge port, an S-100 backplane, or an Apple II slot, the **method** is unchanged but specific things move. Use this as the diff.

Connector and voltage	A new adapter PCB (Ch. 6); the level-shifting strategy depends on the bus voltage and drive.
Signal-to-GPIO mapping	If the bus does not match the ISA-shaped header, the adapter re-wires it (as CoCo/MSX do) — and the PIO `.define`s follow.
The decode rule	`wait_sel` in the `.pio` (Ch. 9): which lines mean "selected", and the timing reference (a clock edge, a chip-select, or a strobe).
Data-direction timing	The `read` program: what edge to flip `pindirs` on. Synchronous buses hang it on the clock; asynchronous ones on the strobe.
Byte-pipe address	`IO_BASE` / `BUS_ROM_BASE` and the register offsets — wherever the host can reach four registers and a ROM.
Host loader + screens	`fujinet-config/src/<platform>/` — the loader's access method and the CONFIG character set.
Client port I/O	`fujinet-lib` `bus/<platform>/portio.*` — memory-mapped or port-mapped reads/writes.

Table 21. The per-bus diff. Everything else — FujiBus framing, the ESP32 device and media classes, the `N:` protocol stack, the byte-pipe model — is shared and unchanged.

> **TIP**
> The fastest possible bring-up of a brand-new bus is: feed the PIO a single decoded "card selected" line from a GAL on the adapter (so `wait_sel` reduces to the MSX form), put the byte pipe wherever the host can do four `peek` / `poke`s, and reuse `BUILD_RS232` whole. You can have a machine talking to FujiNet before you have written one line of decode logic in PIO.

APPENDIX A

FujiBus quick reference

Frame: `END` (`0xC0`) · header · descriptors · AUX (little-endian) · payload · `END` . SLIP escapes: `0xC0` → `0xDB 0xDC` , `0xDB` → `0xDB 0xDD` .

Header (6 bytes): `device` , `command` , `length` (u16 LE, total incl. header), `checksum` (add-with-carry-fold, this byte zeroed during calc), `descr` (first field descriptor).

Field descriptors (`descr & 0x07` ; bit 7 = more follow):

0 none	1 1×u8	2 2×u8	3 3×u8	4 4×u8	5 1×u16	6 2×u16	7 1×u32

Replies: `command` = `ACK` `0x06` (success, optional payload) or `NAK` `0x15` (failure).

Device IDs: `0x31` – `0x3F` disk · `0x40` – `0x43` printer · `0x45` clock · `0x50` – `0x53` serial · `0x5A` CP/M · `0x70` Fuji control · `0x71` – `0x78` network · `0x99` MIDI · `0xFF` bus controller (DBC, intercepted by the RP2350).

Common commands (`fujiCommandID.h` is authoritative; high range is the Fuji control device):

`0x4F`	`'O'`	`OPEN`	`0xF8`	`MOUNT_IMAGE`
`0x52`	`'R'`	`READ`	`0xF9`	`MOUNT_HOST`
`0x57`	`'W'`	`WRITE`	`0xF7`	`OPEN_DIRECTORY`
`0x53`	`'S'`	`STATUS`	`0xF6`	`READ_DIR_ENTRY`
`0x43`	`'C'`	`CLOSE`	`0xE8`	`GET_ADAPTERCONFIG`
`0x50`	`'P'`	`PARSE` / `PUT`	`0xD9`	`CONFIG_BOOT`
`0x51`	`'Q'`	`QUERY`	`0x80`	`JSON_PARSE`
`0x06`		`ACK`	`0x81`	`JSON_QUERY`
`0x15`		`NAK`	`0xFF`	`RESET`

Table 22. A working subset of FujiBus commands. The full enum (≈150 names, with per-device aliases) is `fujiCommandID.h` , shared by firmware and lib.

APPENDIX B

ISA 8-bit (PC/XT) 62-pin pinout

The standard 8-bit ISA edge, with the universal board's GPIO assignment for the signals FujiNet uses (from `tbl-gpmap`). Pin rows: A = component side, B = solder side.

A1	`/I/O CH CK`	—
A2–A9	`D7` … `D0`	`27` … `20`
A10	`I/O CH RDY`	—
A11	`AEN`	`32`
A12–A31	`A19` … `A0`	`19` … `0`

Table 23. A side (data, address, `AEN`, ready).

B1	`GND`	—
B2	`RESET DRV`	`35`
B3	`+5V`	—
B11	`/SMEMW`	`29`
B12	`/SMEMR`	`28`
B13	`/IOW`	`31`
B14	`/IOR`	`30`
B20	`CLK`	`33`
B28	`ALE`	`34`
B30	`OSC`	—
B4..B26	`IRQ2-7`, `DRQ/DACK`	—
B5/B7/B9	`-5V` / `-12V` / `+12V`	—

Table 24. B side (power, strobes, clocks, IRQ/DMA).

> **NOTE**
> The schematic's net names confirm this mapping: `~{SMEMR}` / `~{SMEMW}`, `~{IOR}` / `~{IOW}`, `AEN`, `ALE`, `CLK`, `OSC`, `IO_READY`, `IRQ2` – `IRQ7`, `DRQ1` – `DRQ3`, `~{DACK0}` – `~{DACK3}`, `TC`, and `BA00` – `BA19` (buffered address). FujiNet uses only the boxed subset above.

APPENDIX C

Universal board jumper & test-point reference

`JP1` – `JP36`	solder, bridged	One per bus signal; cut to isolate or to insert a buffer.
`JP37` – `JP39`	solder, open	Optional configuration straps.
`J1`	`Bus_ISA_8bit`	Universal bus header — adapter mates here.
`J2`	2×13 header	ISA-side breakout (logic-analyzer tap).
`J3` , `J4`	2×13 / headers	RP2350 GPIO breakouts.
`J5` – `J8`	1×20	Core2350B and ESP32-S3 module seats.
`J9`	2×2	Power-source selection.
`U1`	module	WaveShare Core2350B (RP2350B).
`U2`	module	Freenove ESP32-S3-CAM.
`D1`	diode	Power-rail protection.

Table 25. Reference designators on `Universal-proto-v1` . There are no buffer ICs; add level-shifting on the adapter (Ch. 6).

APPENDIX D

Repository map

Where every artifact in this guide lives.

`fujiversal/main.cpp`	Core 0 USB bridge + core 1 `romulan()` bus loop + DBC handler.
`fujiversal/boards/*.pio`	Per-board PIO + pin defines + `BusSignals` union (add `isa_proto.pio`).
`fujiversal/setup_sm.cpp`	Generic PIO state-machine setup helper.
`fujiversal/FujiBusPacket.*`	FujiBus encoder/decoder (RP2350 copy).
`fujiversal/CMakeLists.txt`	Board selection, `PICO_PIO_USE_GPIO_BASE`.
`fujiversal-pcb-prototype/Bus-proto/`	`Universal-proto-v1` board (jumpers, breakouts).
`fujiversal-pcb-prototype/*-adapter/`	CoCo / MSX adapters (templates for the ISA adapter).
`fujiversal-pcb-prototype/parts.pretty/ISA_8bit*`	ISA edge footprints.
`fujinet-firmware/build-platforms/platformio-fujiversal-*.ini`	Build targets (add `…-isa.ini`).
`fujinet-firmware/lib/bus/rs232/`	FujiBus transport + `systemBus` (reused as-is).
`fujinet-firmware/lib/device/rs232/`	Device classes (Fuji, disk, network, printer, …).
`fujinet-firmware/lib/media/`	Image formats (`MediaType`).
`fujinet-firmware/include/pinmap/`	Pin maps (add `fujiversal_isa.h`).
`fujinet-lib-experimental/bus/<plat>/`	Per-platform transport + port I/O (add `bus/isa/`).
`fujinet-lib-experimental/common/`	Shared `network`, `json`, `fuji` code.
`fujinet-config/src/<plat>/`	Host loader + CONFIG screens (add `src/isa/`).

Table 26. The file map. “Add …” marks the artifacts a new ISA platform creates; everything else is reused.

APPENDIX E

Glossary

AEN Address Enable. High during ISA DMA; an I/O card must decode only when it is low.

Byte pipe the four I/O registers (`GETC` , `STATUS` , `PUTC` , `CONTROL`) through which the host streams bytes to and from the RP2350.

DBC device ID `0xFF` , the bus controller (RP2350) itself; DBC-addressed packets are consumed locally for ROM bank-switching.

FEP-004 the FujiNet protocol proposal that FujiBus implements.

FujiBus the SLIP-framed packet protocol the host and ESP32 exchange.

fujiversal the RP2350 firmware that emulates the bus interface.

Option ROM a BIOS-scanned expansion ROM (`0x55 0xAA` header) in `0xC0000` – `0xDFFFF` ; FujiNet's boot loader on ISA.

PIO the RP2350′s Programmable I/O — deterministic state machines that implement the bus timing.

RAMROM the RP2350′s swappable emulated-ROM image, bank-switched via DBC commands.

Tandem design the ESP32 + RP2350 pairing for bus-based platforms.

APPENDIX F

Bill of materials (one bring-up rig)

1	Waveshare Core2350B (RP2350B)	`U1`; ≥ 40 usable GPIO
1	Freenove ESP32-S3-CAM (dual-USB, microSD)	`U2`; may need SD-pin ground mod
1	Universal-proto-v1 PCB + headers	`fujiversal-pcb-prototype`
1	ISA adapter PCB	you fabricate (Ch. 6)
1–2	`74LVC245` (data) + `74LVC` buffers (addr/strobe)	for the buffered adapter
1	GAL/ATF16V8 (optional)	card-select decode for `wait_sel`
—	0.1 µF + 10 µF decoupling, headers, jumper wire	
1	USB hub with per-port power	reflash without disturbing the bus
1	Logic analyzer (≥ 8 ch, ≥ 24 MS/s)	`J2` / `J3` / `J4` probing
1	ISA slot extender or socketed breakout	saves the card-edge gold during bring-up

Table 27. What it takes to bring up one tandem ISA board. The two dev boards and the proto board reach milestone 3 on their own.

FujiNet Platform Bring-Up Guide — Revision 1, June 2026.
Built with Typst from sources in `fujiversal`, `fujiversal-pcb-prototype`, `fujinet-firmware`, `fujinet-lib-experimental`, and `fujinet-config`.
The network is as easy as the disk drive — once the bus says so.